

UNIT-2-ARCHITECTURAL DETAILS AND ASSEMBLY LANGUAGE

PROGRAMMING OF 8085 MPU

- 2.1 Internal Architecture & Description
- 2.2 Instruction Classification
- 2.3 Addressing modes
- 2.4 Instruction Cycle, Machine Cycle, t-state, Timing Diagram, RTL
- 2.5 Basic Assembly language programs of 8085.

MICROPROCESSOR: 8085

ARCHITECTURE.

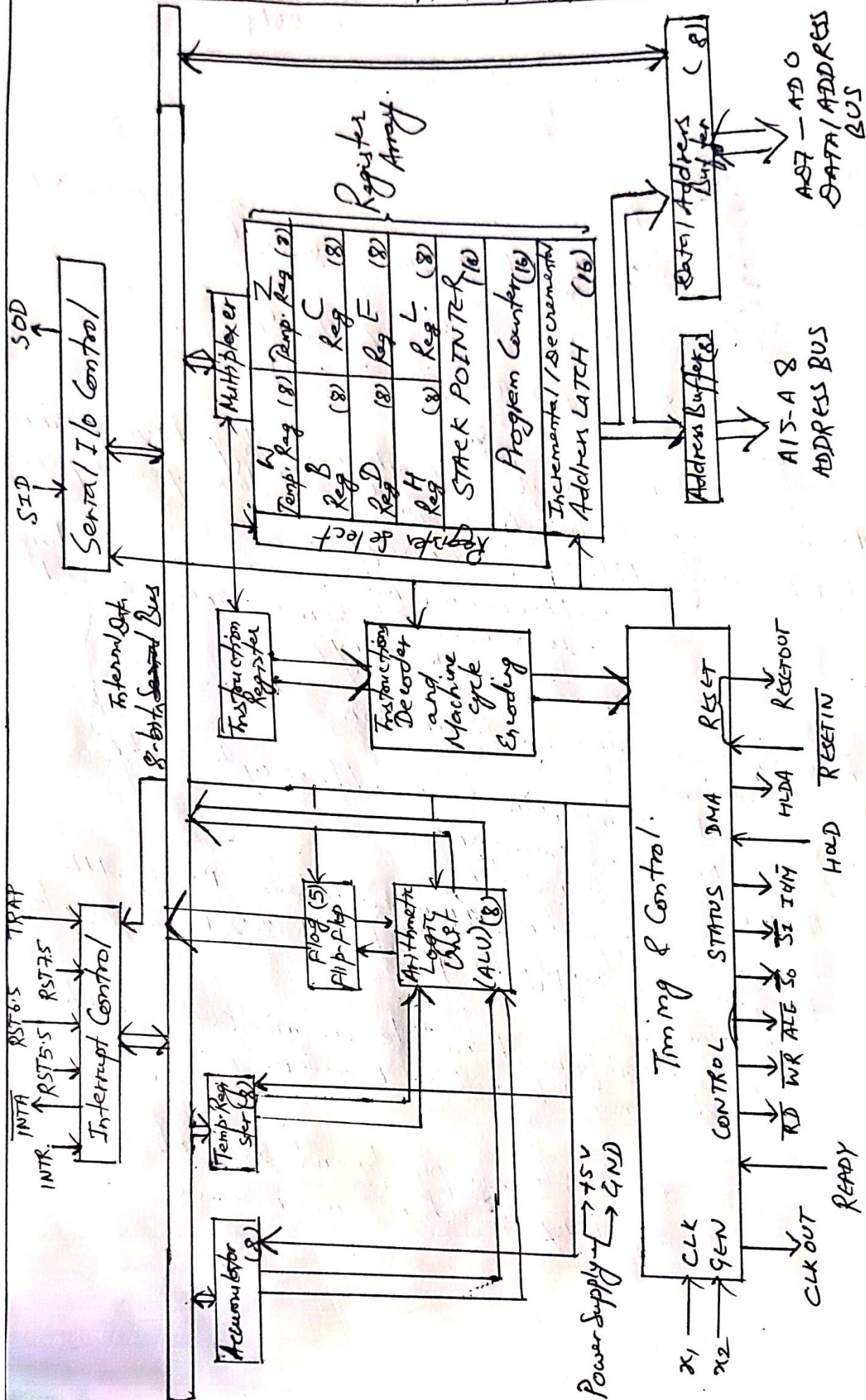
Introduction: Pronounced as "eighty-eighty five" microprocessor. It is an 8-bit microprocessor developed by Intel in 1977 using NMOS technology.
↳ (N-channel metal oxide semiconductor)
< is a microelectronic circuits used for logic & memory >.

It has the following configuration:

- 8-bit data bus
- 16-bit address bus, which can address upto 64 kB.
- 16 bit program counter
- A 16 bit stack pointer
- six 8-bit registers arranged in pairs: BC, DE, HL
- Requires +5V supply to operate at 3.2 MHz single phase clock.
- Other key components include accumulator, instruction registers, control bus, flags register.

Popular choice due to its simplicity and ease of use.
It is used in washing machines, microwaves ovens, mobile phones etc.

< BLOCK DIAGRAM OF 8085 MPU >



8085 Microprocessor - functional Units

- ① Accumulator: It is an 8-bit register used to (perform) arithmetic, logical, I/O and LOAD/STORE operations & store results. It is connected to external data bus and ALU.
- ② Arithmetic & Logic Unit: It performs arithmetic & logical operations like addition, subtraction, AND, OR etc.
- ③ General Purpose Register: There are 6 general purpose registers in 8085 processor i.e., B, C, D, E, H & L. Each hold 8 bit data. They also can work in pair to hold 16-bit data. Commonly used to store memory addresses.
- ④ Program Counter: It is 16 bit register used to store the memory address location of the next instruction to be executed. The program counter is incremented after each instruction is executed, which allows the microprocessor to execute instructions in sequence.
- ⑤ Stack Pointer: 16-bit register, works like stack. Keep track of the top of the stack.
- ⑥ Temporary Register: It is an 8 bit register, which holds the temporary data of arithmetic & logical operations.
- ⑦ Flag Register: It is an 8-bit register having five 1-bit flip flops, which holds either 0 or 1 depending upon the result stored in the accumulator.
 - 5 flip flops set are
 - ① Sign (S)
 - ② Zero (Z)
 - ③ Auxiliary Carry (AC)
 - ④ Parity (P)
 - ⑤ Carry (C)

Its bit position is shown in following table:

D7	D6	D5	D4	D3	D2	D1	D0
S	Z		AC		P		CF

⑧ Instruction Register & Decoder: When the instruction is fetched from the memory then it is stored in the instruction register. Instruction decoder decodes the information present in the instruction register.

⑨ Timing & Control Unit: Provides timing and control signals to the microprocessor to perform operations. It comes under CPU section. It controls the flow of data from CPU to other devices. Example:- RESET signals, Status signals etc.

- Control Signals: Ready, RD, WR, ALE

- Status Signals: SO, SI, IO/M

- DMA Signals: HOLD/HLDA

- RESET Signals: RESET IN, RESET OUT

- * SO & SI are output status signals used to give information of operation performed by microprocessor.

- * HOLD/HLDA: HOLD signal indicates that another master is requesting the use of the address and data buses.

HLDA - HOLD Acknowledge - indicates that CPU has received the HOLD request & it will relinquish the bus in the next clock cycle.

⑩ Interrupt Control: Whenever the interrupt occurs, the microprocessor shifts the control from the main program to the process the incoming request. After completion, the control goes back to main program. 5-INTR, RST 7.5, RST 6.5, RST 5.5, TRAP

⑪ Serial Input/Output Control: It controls the serial data communication by using two instructions: SID (Serial input data) and SOD (Serial output data).

⑫ Address Buffer & Address Data Buffer: The content stored in the stack pointer and program counter is loaded into the address buffer & address data buffer to communicate with the CPU. The memory & I/O chips are connected to these buses.

- ③ Address Bus & Data Bus: Data bus carries the data to be stored. It is bidirectional whereas address bus is unidirectional. It carries location (address) to where data should be stored.

THE FLOW OF AN INSTRUCTION IN 8085 ARCHITECTURE:

- 1) Execution starts with Program Counter. It starts the program execution with next address field. It fetches the instruction from the memory location pointed by Program Counter.
- 2) For address fetching from memory, multiplexed address/data bus acts as an address bus and after fetching instruction this address bus will now act as a data bus & extract data from the specified memory location & send this data on an 8-bit internal bus. For multiplexed address/data bus Address Latch Enable (ALE) Pin is used.
↓
(allows the input address to be entered to a register)
• If $ALE=1$, multiplexed bus is address bus otherwise it acts as Data bus.
- 3) After data fetching, it will go into the instruction register. It will store data fetched from memory & data is now ready for decoding. (instruction decoder is used)
- 4) After that, timing & control comes into the account. It sends control signals to all over the microprocessor to tell microprocessor whether the given instruction is for READ/WRITE & whether it is for MEMORY/I/O Devices activity.
- 5) According to the timing and control signal pins, logical and arithmetic operations are performed (ALU).
- 6) With the help of Serial I/O data pin (SID & SOD) we can send or receive input/output to external devices. In this way, execution cycle is carried out.

7). While execution is going on if there is any interrupt detected then it will stop execution of the current process and invoke (ISR) Interrupt Service Routine function, which will stop the current execution & do the execution, current occurred interrupt.

USES OF MICROPROCESSOR-8085

- Embedded Systems.
- Computer Peripherals.
- Communication System.
- Instrumentation & Control Systems.
- Home Appliances.
- Educational Purposes. etc.

ISSUES IN 8085 MICROPROCESSOR

- Overheating
- Power Supply Issue
- Timing Issues
- Hardware Interface Issues
- Programming Issues
- Research & Development
- Retro Computing

8085 PIN DESCRIPTION:

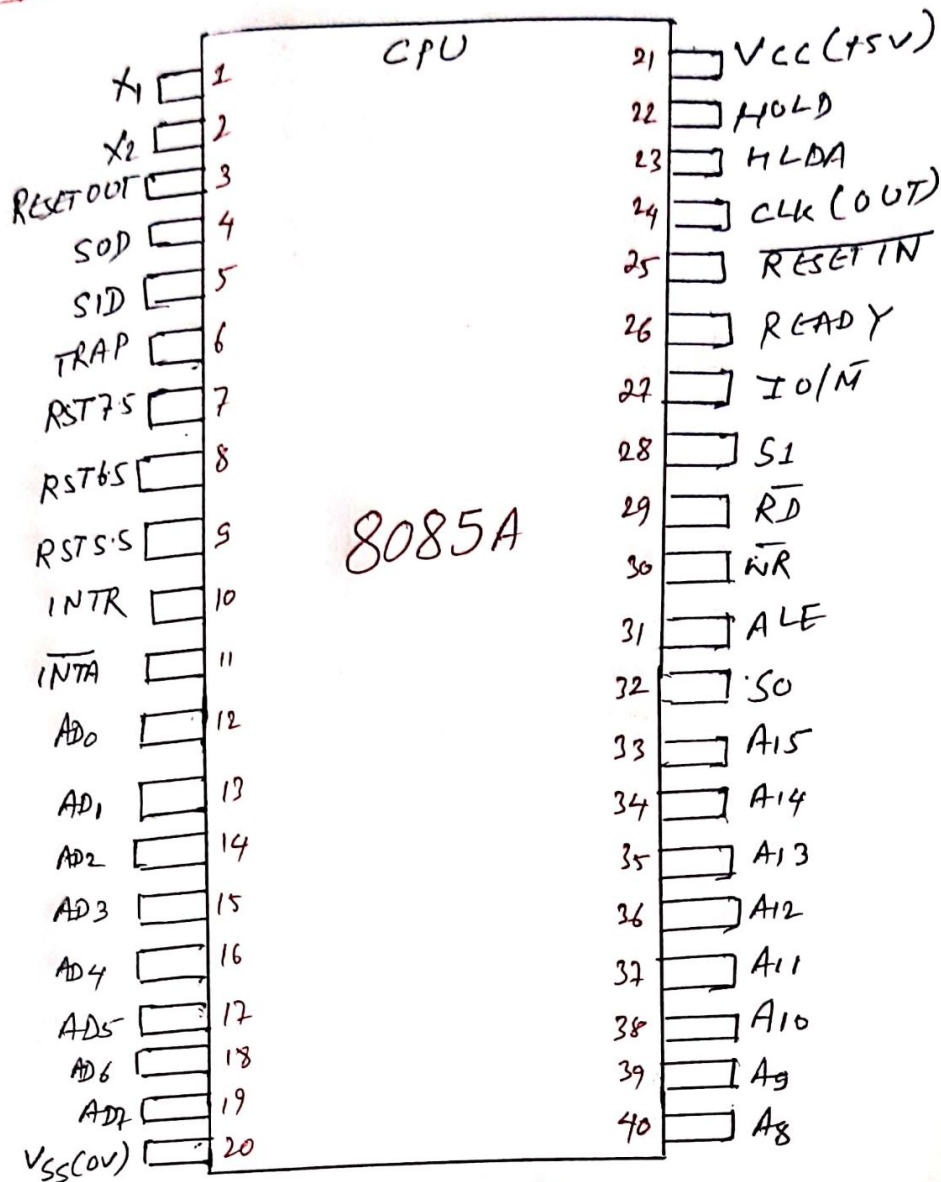


Fig: 8085A Pin Diagram

- 1) It is 8 bit microprocessor. ALU can perform only 8 bit operation at a time.
- 2) 40 pins present. It is a single chip.
- 3) Works on 5 volts.
- 4) Maximum clk frequency 3 MHz & min 500 kHz.
- 5) 74 instructions based on addressing modes.
- 6) Multiplexed address and data. ($AD_0 \dots AD_7$).
- 7) 8085 provides 16 address line and 8 data line.
- 8) 256 Software interrupt

2.2.5 Instruction Classification

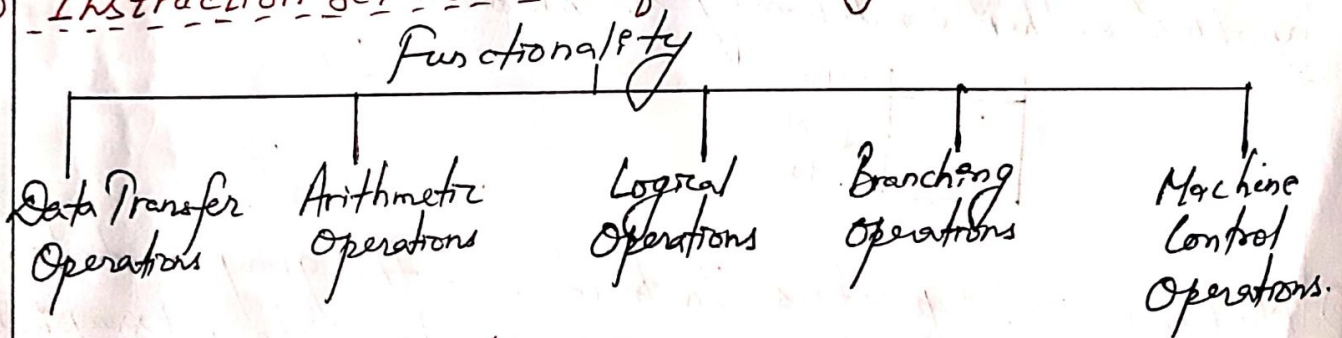
① What is an Instruction?

- An instruction is a binary pattern designed inside a microprocessor to perform a specific function.
- The entire set of instructions that a microprocessor supports is called Instruction Set.
- Based on the ALU and decoding unit, the manufacturer provides instruction set for every microprocessor.
- The instruction set consists of both machine codes and Mnemonics.
 - Machine codes → code defined by the manufacturer for each function.
 - Mnemonics → binary code, easy to understand. Ex. ADD, SUB etc.
- Program: Set of instructions written by the user to perform certain task.

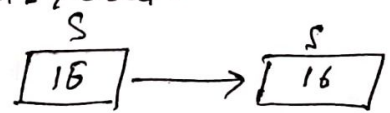
Microprocessor instruction can be classified based on

- ① Functionality
- ② Length
- ③ Operand Addressing

① Instruction Set based on functionality



① Data Transfer Operations: This group of instructions copies data from source to destination. The content of the source is not altered.

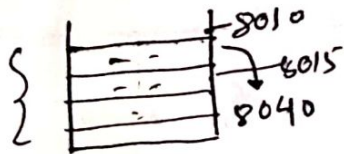


- ② **Arithmetic Operations:** Instructions of this group perform operations like addition, subtraction, increment & decrement. One of the data used in arithmetic operation is stored in accumulator and the result is also stored in accumulator.
- ③ **Logical Operations:** Logical operations include AND, OR, XOR ^{2 operands} and NOT.
One operand is stored in accumulator, and "other operand" can be any register or memory.
NOT - Single operand is required
↓
stored in accumulator.

OP CODE: operation Code: It is the instruction that is executed by the CPU.

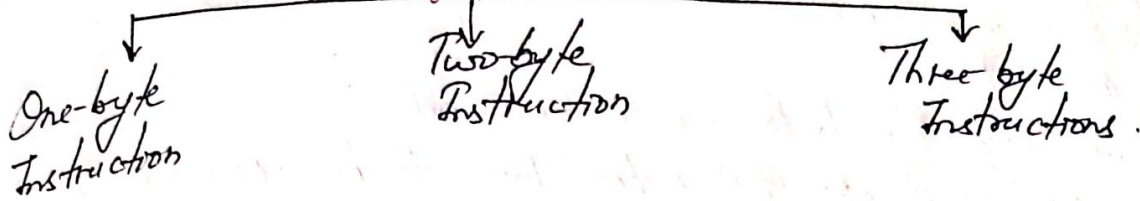
OPERAND: An operand is the second part of the instruction, which tells the computer where to find or store the data or instructions.

- ④ **Branching Operations:** Instructions in this group can be used to transfer program sequence from one memory location to another either conditionally or unconditionally.



- ⑤ **Machine Control Operations:** Instructions in this group control execution of other instructions & control operations like interrupt, halt etc.

Instruction Set Classification based on Length:



Eg: of One-byte Instruction.

MOV A, B - moving contents of Register B into the register A.
 machine code - 78 - when 78 is stored in micro processor, it will occupy only one-byte.

A 88 M 86 - 1 byte

Eg: of Two-byte Instruction

MVI A, 7F H
 ↓
 data

machine code for MVI → 3E 7F } 2 byte

↓
 Move immediate data

ADI OF, H

06 0F } 2-byte

Eg: of Three byte Instruction

JMP 9050 H
 higher ↓ lower order address

machine code for jmp → C3 50 90 } 3-byte

LDA 8850 H

3A 50 88 } 3-byte

1000 1000 0101 0000 → 16 bits

8-bits → 1 byte

③ Instruction Set Classification based on Operand Addressing:

• Opcode
(task to be performed by the instruction)

• Operand
(data to be operated)

• Various ways for defining the operand.

* The process of specifying the data to be operated on (operand) by the instruction is called ADDRESSING.

→ The various formats for specifying operands are called Addressing Modes.

5 types of Addressing Modes:

- Immediate Addressing Mode
- Memory Direct Addressing Mode
- Register Direct Addressing Mode
- Indirect Addressing Mode
- Implicit Addressing Mode.

a) Immediate Addressing Mode: In this mode, the operand is given in the instruction. A byte or word which transfers to the destination register or the memory location.

Eg: MVI A, 9A H

A [9A]

↑ immediate data. (instruction itself)
↘ operand

- ① operand is a part of the instruction
- ② operand is stored in the register mentioned in the instruction.

b) Memory Direct Addressing Mode: In this mode, a byte or word is moved between a memory location and register. The memory location is given in the instruction.

Ex: LDA 850F H

load the accumulator with contents of memory location given in [850F] H

A [20]

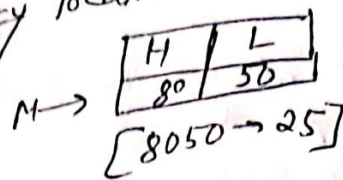
850F [20]

1) Register Direct Addressing Mode: In this mode, the address of the register is given in the instruction itself.

Ex: MOV B, C - transferring between 2 registers.

2) Indirect Addressing Mode: In this mode, the data is either stored in the memory location pointed by a register pair.

Eg: MOV ^{accumulator} A, M ← memory location.



HL pair register.

3) Implicit Addressing Mode: In this mode, the data itself specifies the data to be operated upon.

Eg: CMA
complement the contents of Accumulator.

A [23]

① DATA TRANSFER GROUP: This group consists of following sets of instructions.

a) MOV r, data (Move data immediate to register r)
→ This instruction directly loads a single register with a single byte of data. Example - MVI M, 08.

b) MOV r₁, r₂ - Move content of one register to another.
r₁ ← r₂ . Example MOV A, B.

c) MOV r, m → Move the content of memory to register.
Ex - MOV B, M

d) MOV M, r → move content of register to memory.
Ex: MOV M, C

e) LDA addr → Load a accumulator.
[A] ← [addr] Ex - LDA 2400 H

8) **LHLD addr** [Load H & L Direct].

Ex $\rightarrow$ LHLD 2500H

$\rightarrow$ Copies the contents of memory location given by the address onto the register L & the contents of the next address location onto the Register H.

9) **STA addr** $\rightarrow$ Store accumulator $\rightarrow$ Store data from A
direct at the address that follows.
STA 2000H

10) **XCHG** $\rightarrow$ Exchange contents of register H with D & L with E.

$H \leftrightarrow [H-L] \leftrightarrow [D-E]$.

11) **SHLD addr** (Store H & L direct): Content of the register L is stored in memory location corresponding to the address given & the contents of the register H at the next address location.

② ARITHMETIC GROUP (INSTRUCTIONS):

This group consists of following set of instructions:

i) Addition. (ADD).

ii) **ADD r** (Add register r to A) Ex: $[A \leftarrow A+r]$ ADD R.
The content of specified register are added to the contents of the Accumulator.

iii) **ADD M** (add data in memory to A)
 $[A] \leftarrow [A] + [H-L]$.

iv) **ADDI data** (add data immediate to A).
 $[A] \leftarrow [A] + \text{data}$

v) **ACI data** (Add with carry immediate data to accumulator)
 $[A] \leftarrow [A] + \text{data} + [CS]$

vi) **ADC M** (Add data in memory to A with Carry)
 $[A] \leftarrow [A] + [H-L] + [CS]$

SUBTRACTION (SUB)

- $SUB\ r$ [Subtract register r from A]
 $[A] \leftarrow [A] - [r]$
- ii) $SUB\ M$ [Subtract data in memory from A]
 $[A] \leftarrow [A] - [H-L]$
- iii) $SUBB\ r$ [Subtract register r from A with borrow]
 $[A] \leftarrow [A] - [H-L] - [CS]$
- iv) $SUI\ data$ [Subtract immediate from A].
- v) $SBB\ M$ [Subtract memory data from A with borrow]
 ~~$[A] \leftarrow [A] - [H-L]$~~

INCREMENT & DECREMENT

- a) $INR\ r$ (Increment register) (Increase by 1)
- b) $INR\ M$ (Increment data in Memory) $\rightarrow$ by 1
- c) $INX\ sp$ (Increment register pair) $\rightarrow$ by 1
- d) $DCR\ r$ (Decrement register)
- e) $DCR\ M$ (Decrement data in memory)
- f) $DCX\ sp$ (Decrement Register pair).

DECIMAL ADJUST ACCUMULATOR:

DAA (Decimal Adjust A).

BRANCHING INSTRUCTIONS:

- conditional or unconditional jump, subroutine call, return and restart.
- Unconditional Jump
 $JMP\ address$ [Jump to instruction specified by the address]
 $[PC] \leftarrow label$

• Conditional JMP
 a) Jump addr (label) → jump to instruction specified by the address if the specified condition is fulfilled.
 $[PC] \leftarrow \text{label}$

b) JZ addr [label] → Jump if result zero.
 $[PC] \leftarrow \text{address [label]}$

c) JNZ addr [label] → Jump if the result is not zero.
 $[PC] \leftarrow \text{address [label]}$

d) JC addr [Jump if there is carry]

e) JNC addr [Jump if there is no carry]

f) JP addr [Jump if the result is plus]

g) JM addr [Jump if the result is minus]

h) JPE addr [Jump if even parity]

i) JPO addr [Jump if odd parity]

• Rstart [RST] - one word call instruction.

④ LOGICAL INSTRUCTIONS:
 It consists of following instructions: (AND, OR, compare, rotate etc)

i) ANA r (AND Register with accumulator).
 It logically ANDs the content of specified register with those of the Accumulator bit by bit & places the result in the accumulator.

ii) ANA M (AND Memory contents with A) → memory location whose address is in H.L memory with accumulator)

iii) ANI data (AND immediate with A)

iv) XRA r (Exclusive OR register with A)

v) ORA r (OR register r with A)

vi) ORA M (OR memory contents with A).

vii) CMP r (Compare Register with A).

164
2/23/5
CMP M (Compare Memory data with A).

ROTATE

RLC $\rightarrow$ (Rotate A to left)

RRC $\rightarrow$ (Rotate A to right)

RAL $\rightarrow$ (Rotate A left through carry)

RAR $\rightarrow$ (Rotate A right through carry)

③ MACHINE CONTROL INSTRUCTIONS:

[Stack, I/O & Machine Control Group]

- ① IN Port - Address $\rightarrow$ Input to Accumulator from I/O port.
[A] $\leftarrow$ [Port]
- ② OUT Port - Address $\rightarrow$ Output from Accumulator to I/O port.
[Port] $\leftarrow$ [A]
- ③ PUSH sp $\rightarrow$ Push the content of register pair to stack.
- ④ PUSH PSW $\rightarrow$ Push processor word.
- ⑤ POP sp (Pop Register pair off the stack)
- ⑥ XTHL (Exchange top of stack with H & L)

I/O instruction. (Already mentioned above)

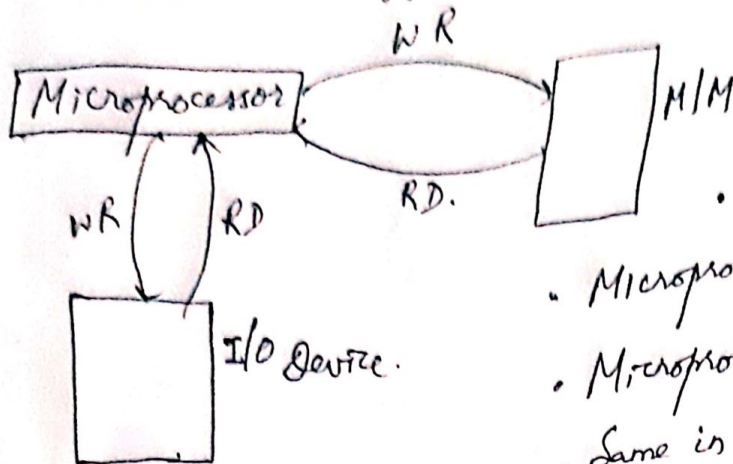
Machine Control:

0. EI $\rightarrow$ Enable Interrupts.
1. DI $\rightarrow$ Disable Interrupts.
2. NOP $\rightarrow$ No operation.
3. HLT $\rightarrow$ Halt the processor.
4. SIM $\rightarrow$ (Set Interrupt Mask).
5. RID $\rightarrow$ Read Interrupt Mask.

2.4 INSTRUCTION CYCLE, MACHINE CYCLE, ~~T-state~~, TIMING DIAGRAM, RTL.

- The instruction cycle of 8085 microprocessor consists of 4 basic steps.
 - ① Fetch: Fetches the instruction from the memory location pointed to by the program Counter (PC) ✓
 - ② Decode: Microprocessor decodes the instruction to determine the operation to be performed.
 - ③ Execute: Microprocessor performs the operation specified by the instruction on the operands.
 - ④ Store: Result stored in the appropriate memory location or register.
- * The cycle repeats until the program is complete or interrupted.
- * Why do you need instruction cycle in 8085 microprocessor?
- It is fundamental concept in the operation of 8085 because it is the process by which the microprocessor fetches, decodes and executes instructions.
 - The execution of a program in a microprocessor involves a sequence of instructions & each instruction is executed using the instruction cycle.
 - The execution should be in correct sequence and correct operation should be performed.
 - Instruction cycle allows the microprocessor to execute instructions at a very high speed.
 - "Time required to execute & fetch an entire instruction is called instruction cycle."
 - "The time required by the microprocessor to complete an operation of accessing memory or input/output device is called machine cycle."

One time period of frequency of microprocessor is called t-state. (trigger state)



- up is a brain.
- Microprocessor $\xrightarrow{\text{Write operation}}$ M/M
- Microprocessor $\xleftarrow{\text{Read operation}}$ M/M
- Same in I/O Device.

Name	Address
Op code Latch	416
Memory Read	3
Memory Write	3
I/O Read	3
I/O Write	3

Need to decode

t-state. (38 state)

- ① First generate address.
- ② Address is locked and data is received.
- ③ Stabilize the data.

LDA 2000H	
cycle	T-states
9	13

opcode fetch = 4T
m/m Rd = 3T
m/m Rd = 3T
RD = 3T

MOV A, B	⇒ opcode fetch
cycles	T-states
1	4

Many t-state → Machine cycle
↓
Instruction cycle.

★ TIMING DIAGRAM:

Definition: Timing diagram is a graphical representation. It represents the execution time taken by each instruction in a graphical format. The execution time is represented in T states.

- 8085 microprocessor has 5 basic machine cycle.
- ① opcode fetch cycle (4T)
- ② Memory Read cycle (3T)
- ③ Memory Write cycle (3T)
- ④ I/O Read cycle (3T)
- ⑤ I/O Write cycle (3T)